

SKILL - WEEK 8

Task -1:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with combined redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">");

    if (redirect == NULL)
    {
        printf("No output redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 1;

    if (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;

    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    char *error_redirect = strstr(filename, "2>&1");

    if (error_redirect != NULL)
    {
        *error_redirect = '\0';
        end = filename + strlen(filename) - 1;

        while (end >= filename && *end == ' ')
        {
            *end = '\0';
            end--;
        }
    }

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("File open");
        return 1;
    }

    int saved_stdout = dup(STDOUT_FILENO);
    int saved_stderr = dup(STDERR_FILENO);

    if (fork() == 0)
    {
        dup2(fd, STDOUT_FILENO);

        if (error_redirect != NULL)
            dup2(STDOUT_FILENO, STDERR_FILENO);

        close(fd);

        char *args[20];
        int i = 0;
        char *token = strtok(command, " ");

        while (token != NULL && i < 19)
        {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    wait(NULL);

    dup2(saved_stdout, STDOUT_FILENO);
    dup2(saved_stderr, STDERR_FILENO);

    close(saved_stdout);
    close(saved_stderr);
    close(fd);

    printf("Combined redirection completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
jagadeesh-14@Jagadeesh:~$ nano combined_redirect.c
jagadeesh-14@Jagadeesh:~$ gcc combined_redirect.c -o combined_redirect.c
gcc: fatal error: input file 'combined_redirect.c' is the same as output file
compilation terminated.
jagadeesh-14@Jagadeesh:~$ gcc combined_redirect.c -o combined_redirect
jagadeesh-14@Jagadeesh:~$ echo "This file exists." > sample.txt
jagadeesh-14@Jagadeesh:~$ cat sample.txt
This file exists.
jagadeesh-14@Jagadeesh:~$ ./combined_redirect
Enter command with combined redirection: ls sample.txt nonexistent.txt > output.txt 2>1
ls: cannot access 'nonexistent.txt': No such file or directory
Combined redirection completed successfully.
jagadeesh-14@Jagadeesh:~$ ./combined_redirect
Enter command with combined redirection: ls sample.txt nonexistent.txt > output.txt 2>&1
Combined redirection completed successfully.
jagadeesh-14@Jagadeesh:~$ cat output.txt
cat: output: No such file or directory
cat: txt: No such file or directory
jagadeesh-14@Jagadeesh:~$ cat output.txt
ls: cannot access 'nonexistent.txt': No such file or directory
sample.txt
jagadeesh-14@Jagadeesh:~$ |
```

Task -2 :

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int p[2];
    pipe(p);

    int in = open("input.txt", O_RDONLY);
    int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fork() == 0)
    {
        dup2(in, 0);
        dup2(p[1], 1);

        close(p[0]);
        close(p[1]);

        execlp("cat", "cat", NULL);
    }

    if (fork() == 0)
    {
        dup2(p[0], 0);
        dup2(out, 1);

        close(p[0]);
        close(p[1]);

        execlp("grep", "grep", "Hello", NULL);
    }

    close(p[0]);
    close(p[1]);
    close(in);
    close(out);

    wait(NULL);
    wait(NULL);

    printf("Complex command executed successfully.\n");

    return 0;
}

```

OUTPUT:

```
jagadeesh-14@Jagadeesh: ~  
jagadeesh-14@Jagadeesh:~$ nano complex_command.c  
jagadeesh-14@Jagadeesh:~$ echo "Hello World" > input.txt  
jagadeesh-14@Jagadeesh:~$ gcc complex_command.c -o complex_command  
jagadeesh-14@Jagadeesh:~$ ./complex_command  
Complex command executed successfully.  
jagadeesh-14@Jagadeesh:~$ cat output.txt  
Hello World  
jagadeesh-14@Jagadeesh:~$ |
```